

Services, Auth et APIs - Explication Orale

Projet : WorldCup Hub (Full Stack JS - architecture microservices)

Objectif : expliquer clairement les services, l'authentification (Google + JWT), les deux paradigmes d'API et les bases de données pendant la soutenance.

1. Les services

Le projet contient 4 applications indépendantes, chacune avec son package.json :

frontend/	Next.js (interface)	:3000
main-backend/	Express (gateway + auth)	:4000
services/users-service/	Express + PostgreSQL	:4001
services/favorites-service/	Express + MongoDB	:4002

1.1 main-backend (le gateway)

Fichier principal :

main-backend/src/index.js

Ce que fait le service :

1. Configure CORS (seul le frontend est accepté)
2. Monte les routes /auth (login Google, JWT) et /api/v1 (API métier)
3. Attache le serveur WebSocket /ws/live
4. Oriente chaque requête vers le bon microservice ou l'API externe
5. Gestion d'erreurs centralisée (JSON propre, jamais de stacktrace)

Pourquoi c'est utile :

C'est le point d'entrée unique de l'application. Le frontend ne connaît que lui. Toute la sécurité publique (JWT) est centralisée ici.

Phrase à dire à l'oral :

The main backend is our API gateway: it is the only service exposed to the outside. It handles Google login, issues and verifies JWTs, and routes requests to the two microservices and to the external ML API.

1.2 users-service (PostgreSQL)

Fichiers :

services/users-service/src/index.js
services/users-service/src/db.js

Ce que fait le service :

1. Crée la table users au démarrage (CREATE TABLE IF NOT EXISTS)
2. POST /users/upsert : crée l'utilisateur au premier login Google (INSERT ... ON CONFLICT DO UPDATE sur google_id unique)
3. GET /users/:id : lit un profil
4. Refuse tout appel sans le header x-internal-token (403)

Pourquoi c'est utile :

Un seul service est responsable des comptes. La contrainte d'unicité sur google_id garantit qu'un même compte Google ne crée jamais de doublon.

Phrase à dire à l'oral :

This microservice owns the user accounts in PostgreSQL. The gateway upserts the user at every Google login, so the first login creates the account and the next ones just refresh the profile.

1.3 favorites-service (MongoDB)

Fichiers :

```
services/favorites-service/src/index.js
services/favorites-service/src/models.js
```

Ce que fait le service :

1. Deux collections Mongoose : Favorite et PredictionHistory
2. GET/POST/DELETE /favorites : équipes favorites (index unique userId+team, renvoie 409 si doublon)
3. GET/POST /history : historique des prédictions (avec l'objet probabilities complet renvoyé par le modèle)
4. Refuse tout appel sans le header x-internal-token (403)

Pourquoi c'est utile :

Les données sont des documents flexibles (probabilities est un objet imbriqué) : MongoDB est adapté, pas besoin de jointures.

Phrase à dire à l'oral :

This microservice stores favorites and prediction history in MongoDB. We chose a document database because predictions embed a flexible probabilities object and we never need joins.

2. L'authentification

2.1 Login Google (OAuth 2.0)

Fichier :

```
main-backend/src/routes/auth.js
```

Ce que fait le flux :

1. GET /auth/google : redirection vers l'écran de consentement Google
2. L'utilisateur accepte -> Google renvoie un code temporaire
3. GET /auth/google/callback : échange du code contre le profil (l'app ne voit JAMAIS le mot de passe Google)
4. Upsert de l'utilisateur dans users-service
5. Signature d'un JWT (12 h) et redirection vers le frontend

Pourquoi c'est utile :

Aucun mot de passe à stocker ni à sécuriser : Google porte la charge de l'authentification. Le mode test limite l'accès aux emails déclarés.

Phrase à dire à l'oral :

We use the OAuth 2.0 authorization code flow. Our app never sees the user's Google password: it exchanges a temporary code for the profile, creates the user, and issues its own JWT.

2.2 Le JWT

Fichier :

```
main-backend/src/auth/jwt.js
```

Ce que fait le module :

1. signToken(user) : signe { sub, email, name } avec JWT_SECRET, expiration 12 h
2. requireAuth : middleware qui lit "Authorization: Bearer <token>", vérifie la signature, remplit req.user
3. Token absent ou invalide -> 401 { "error": "Access denied" }

Pourquoi c'est utile :

Le serveur ne stocke aucune session : tout est dans le token signé.
Le payload est lisible (base64) mais infalsifiable grâce à la signature HMAC-SHA256 - modifier le payload invalide le token.

Phrase à dire à l'oral :

The JWT is signed, not encrypted. The server stores no session: it just verifies the signature on every request, which makes the API stateless and easy to scale.

2.3 Le token interne entre services

Fichier :

```
main-backend/src/services/internal.js
```

Ce que fait le mécanisme :

1. Le gateway ajoute le header x-internal-token à chaque appel interne
2. Les microservices comparent ce header à INTERNAL_API_TOKEN (.env)
3. Appel direct sans le secret -> 403 Forbidden

```
$ curl http://localhost:4001/users/1
HTTP 403 { "error": "Forbidden: internal service" }
```

Pourquoi c'est utile :

Défense en profondeur : même si quelqu'un atteint le réseau des microservices, il ne peut rien faire sans le secret partagé.

Phrase à dire à l'oral :

Microservices are not reachable directly: they require an internal shared secret that only the gateway knows. Public JWT security stays centralized in the gateway.

2.4 Protection des routes côté frontend

Fichiers :

```
frontend/app/dashboard/page.js
frontend/lib/api.js
```

Ce que fait la protection :

1. /dashboard vérifie le token au chargement : absent -> redirection /
2. apiFetch ajoute "Authorization: Bearer <token>" à chaque requête
3. Toute réponse 401 (token expiré) purge le token et redirige vers la page de login

Pourquoi c'est utile :

L'utilisateur non connecté ne voit jamais le dashboard. La vraie barrière reste le backend : le frontend n'est que du confort.

Phrase à dire à l'oral :

Protected pages redirect anonymous users to the login screen. But the real security boundary is the backend middleware: frontend checks are only user experience.

3. Les APIs (deux paradigmes + API externe)

3.1 REST - /api/v1

Fichier :

```
main-backend/src/routes/api.js
```

Ce que fait l'API :

POST	/api/v1/predict	prédiction ML + log en historique
GET	/api/v1/matches	matchs de la Coupe du Monde
GET	/api/v1/favorites	liste des équipes favorites
POST	/api/v1/favorites	ajout d'une équipe
DELETE	/api/v1/favorites/:team	suppression
GET	/api/v1/history	historique des prédictions

Toutes exigent un JWT (middleware requireAuth appliqué au routeur).
Erreurs : 401 sans token, 400 paramètres manquants, 502 API externe HS.

Pourquoi c'est utile :

C'est l'exigence "API as a Service" : un client externe (Postman, curl) peut consommer l'API avec un JWT. Démo : un appel avec token -> 200, un appel sans token -> 401 "Access denied".

Phrase à dire à l'oral :

Our REST API is also a service for external consumers: any client with a valid JWT can call it. Without the token the API returns an explicit 401 Access Denied - we can show both calls in Postman.

3.2 WebSocket - /ws/live

Fichier :

main-backend/src/ws/liveScores.js

Ce que fait le WebSocket :

1. Le client se connecte à ws://localhost:4000/ws/live?token=<jwt>
2. Le JWT est vérifié à la connexion (sinon fermeture code 4401)
3. Le serveur récupère les matchs toutes les 60 s et POUSSE la mise à jour à tous les clients connectés
4. Un nouveau client reçoit immédiatement le dernier état connu

Pourquoi c'est utile :

Deuxième paradigme d'API exigé par le sujet. Le serveur pousse au lieu que chaque client re-demande (pas de polling côté client).

Phrase à dire à l'oral :

REST is request/response: the client asks. WebSocket is server push: clients keep one connection open and receive live score updates every minute. The connection itself is authenticated with the same JWT.

3.3 API externe - notre backend MLOps (bonus)

Fichier :

main-backend/src/services/fifaClient.js

Ce que fait le client :

1. POST {FIFA_API_URL}/predict : la prédiction ML (home_team, away_team, stage -> prediction + probabilités)
2. GET {FIFA_API_URL}/matches : les matchs en direct
3. FIFA_API_URL = notre projet MLOps déployé en production sur Render (FastAPI + registre de modèles MLflow)

Pourquoi c'est utile :

Points bonus du sujet : intégration d'un service API externe. Deux projets indépendants (JS ici, Python là-bas) reliés par un contrat HTTP.

Phrase à dire à l'oral :

For the external API integration, we consume our own MLOps project: a FastAPI backend deployed on Render that serves predictions from an MLflow model registry. Two independent systems, one HTTP contract.

4. Bases de données et Docker

4.1 docker-compose.yml (les deux bases)

Fichier :

docker-compose.yml

Ce qu'il fait :

1. postgres:16-alpine sur :5432 (db "users") + volume pgdata
2. mongo:7 sur :27017 + volume mongodata
3. healthchecks sur les deux conteneurs
4. docker compose up -d -> les deux bases démarrent

Pourquoi c'est utile :

Les données ne vivent pas dans les conteneurs (jetables) mais dans des volumes Docker persistants sur la machine hôte. On peut recréer les conteneurs : les données survivent.

Phrase à dire à l'oral :

Both databases run in Docker containers and persist their data in Docker volumes. In production we would switch to managed databases by changing one connection URL - the code does not change (12-factor).

4.2 Dockerfiles des services

Fichiers :

main-backend/Dockerfile
services/users-service/Dockerfile
services/favorites-service/Dockerfile

Ce qu'ils font :

1. Partent de node:22-alpine (image légère)
2. npm ci --omit=dev (dépendances de prod uniquement, reproductible)
3. Copient src/ et exposent le port du service
4. CMD ["node", "src/index.js"]

Pourquoi c'est utile :

Chaque service est déployable indépendamment sur n'importe quelle plateforme (Render, Railway...). La config vient des variables d'environnement, jamais de l'image.

Phrase à dire à l'oral :

Each service has its own Dockerfile, so each one can be deployed and scaled independently. Configuration comes from environment variables, never from the image.

5. Résumé court pour l'oral

Our app is a microservice architecture: a Next.js frontend, an Express gateway, and two microservices with their own databases - users in PostgreSQL, favorites and prediction history in MongoDB.

Authentication uses Google OAuth: the gateway exchanges the Google code for the profile, stores the user, and issues a signed JWT. Every API call and the WebSocket connection require that JWT; microservices also require an internal shared secret, so they are unreachable directly.

We use two API paradigms: REST for CRUD operations and WebSocket for live score push. As a bonus, predictions come from an external API - our own MLOps project deployed on Render, serving an MLflow model.